FrameQuote

Custom Frame Price Calculator & Order Tracker | Semester Project Proposal

Detail	Information
Project members	Syed Hashir Ali, Ibadullah
Project Developer	Syed Hashir Ali (2622-4CS-026)
Project Partner	Ibadullah (2622-4CS-028)
Program	BS Computer Science, Semester 1
University	Al Ghazali University, Ghazali University School of Technology (GU-Tech)
Course	Web Technologies (CS-102)
Submission	31-Oct-2026
GitHub Repository	https://github.com/[your-username]/framequote (Sir added as admin)

This proposal combines two ideas: a custom frame quote and order tracker (customers submit an order and follow its status until delivery) and a frame size and price calculator (customers enter artwork size, choose border and glass, and see an estimated price instantly).

1. Product name

FrameQuote (working name).

2. One-sentence description

My product helps wholesale buyers and shop customers get an instant price for a custom frame and place an order by letting them enter their artwork size, choose a border and glass type, see a calculated estimate, submit the request, and track its status until delivery.

3. The problem

Frame prices and orders are handled manually over WhatsApp and phone calls. The owner must calculate each price by hand, and orders can get lost in long chats. **Example:** A shop owner wants a 30 x 42 inch frame with a crystal border. They message the framing business, wait for the owner to calculate a price, and later have to ask again whether the order is ready.

4. Intended users

Wholesale buyers: gift shops, decorators, and resellers who order several frames of Islamic wall art at once. **Individual customers**: people who want one custom frame for a home or office. **Shop owner / staff (admin)**: manage prices, view orders, and update their status.

5. Current alternatives

People use WhatsApp messages, phone calls, and handwritten notes. These are slow, depend on the owner being available, and make it hard to track orders. FrameQuote gives an instant price at any time, keeps every order in one place, and lets customers check their order status without calling.

6. How the product will work

• The user opens the website and sees the calculator.

• They enter the artwork width and height (inches) and choose a border style, glass type, and quantity.

• The backend calculates the price and the page shows the estimate immediately.

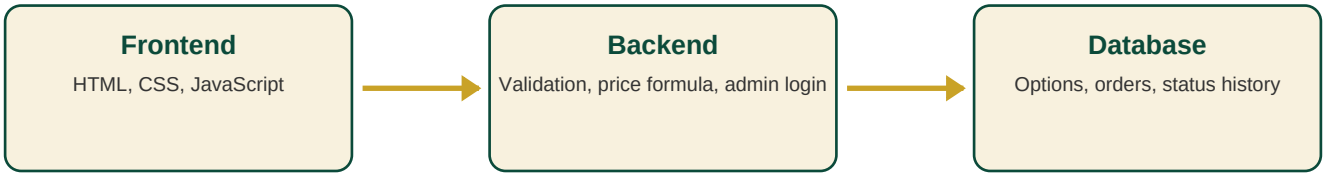
• The user enters their name and phone number and submits the order request.

• The system saves the order and shows an order ID.

• The owner logs in to the admin page, reviews the order, and updates its status: Received, In production, Ready, Delivered.

• The customer checks the status page using their order ID and phone number.

Architecture overview



Calculator, order form, status page, admin page

Hosted database service

7. Ten planned features

#	Feature	What the user can do
1	View frame options	See the available border styles and glass types with their rates.
2	Enter artwork size	Type the width and height of the artwork in inches.
3	Choose options	Select border style, glass type, and quantity.
4	Instant price estimate	See a calculated price before ordering.
5	Submit order request	Send the estimate as an order with name, phone, and notes.
6	Get an order ID	Receive a unique ID after submitting.
7	Track order status	Look up an order using the order ID and phone number.
8	Admin login and order list	Owner signs in and sees all orders, newest first.
9	Update order status	Owner moves an order through its stages.
10	Manage options and prices	Owner adds or edits border styles, glass types, and rates.

8. First version (MVP)

Must work first (3 features): (a) Price calculator (features 2, 3, 4). (b) Submit order request (features 5, 6). (c) Admin order list with status update (features 8, 9).

Can wait: customer order tracking page (7), editing options and prices from the admin page (10; prices can be set directly in the database at first), notifications, and payments.

9. Frontend, backend, and database

Part	What it will do
Frontend	Calculator form, price result card, order form, order status page, admin login page, and admin order list with status buttons. Built with HTML, CSS, and JavaScript (or React).
Backend	Validates inputs (positive sizes, valid phone number), calculates the price, saves orders, generates order IDs, checks admin login, and updates statuses. Formula: price = (area x glass rate + perimeter x border rate) x quantity.
Database	Border styles (name, rate), glass types (name, rate), orders (ID, customer name, phone, width, height, options, quantity, price, status, date), and status history (order, status, time).

10. Example user journey

A gift-shop owner opens FrameQuote and enters a 30 x 42 inch artwork, picks a gold crystal border and clear glass, and sets quantity to 5. The page shows an estimated total. They enter their name and phone number and submit, and receive order ID FQ-1042. The next day, the framing owner logs in, sees the new order, and marks it "In production". Later it becomes "Ready". The shop owner checks the status page with the order ID and phone number and sees that the order is ready for delivery.

11. Privacy and safety

• **Collected:** customer name, phone number, and order details only.

• **Not collected:** no payment details, no customer passwords, no addresses or ID numbers in the first version.

• **Protection:** the status page requires both order ID and phone number, so one customer cannot browse other customers' orders. Only the admin can see the full order list.

• **Admin access:** a safe login method approved by the instructor, with no passwords stored in plain text.

• **Secrets:** database keys are kept in server environment variables, never in frontend code or a public repository.

12. How will you know it works?

#	What will you test?	What result should happen?
1	Enter 30 x 42 inches, pick a border and glass, set quantity 2, and press Calculate.	The estimate matches the formula, and invalid input (zero or negative size) shows an error message.
2	Submit an order request, then refresh the admin page.	The order appears in the admin list with the right details and still appears after refresh.
3	Admin changes an order's status to Ready, then the customer opens the status page.	The customer sees "Ready", and a wrong phone number does not show the order.

13. Deployment and future potential

Deployment: The frontend and backend will be hosted on a free hosting service, and the database on a hosted database service, so the product is available at a live URL. The exact tools (for example Vercel, Render, and Supabase) will be confirmed with the instructor.

Future potential: FrameQuote could be offered to other framing shops and wholesalers who still quote by phone. Before offering it to customers it would need online payments, WhatsApp or SMS notifications, stronger security and backups, invoices, and a way to attach photos of the artwork.

14. What do I need to learn?

1. Building a backend API: routes, input validation, and error handling.

2. Database design and queries: creating tables, relationships, and safe queries.

3. Deployment and security basics: environment variables, admin login, and hosting a live site.

15. AI tools and my learning plan

We will use AI tools to plan the structure, generate first drafts of code, and explain concepts we do not understand yet. We will not paste code we cannot explain. To check generated code, we will:

• run it and test every feature, including wrong inputs and edge cases;

• read each part and ask the AI to explain anything unclear, then re-explain it in our own words;

• keep the project small and build one feature at a time, testing after each step;

• fix errors by reading the error message first and checking the code, not by blindly retrying;

• never share secret keys or private customer data with AI tools or in public code.

We will both be ready to explain the frontend, backend, database, and how they work together during the demonstration.

Git and GitHub workflow

The GitHub repository is shared by both members. Sir (instructor) is added as admin. We will use regular commits with clear messages and pull requests for each feature, so the full project life cycle is visible in the repository history.